

2025 考研数据结构代码题

408 重点代码题

来源：b 站：我头发还多还能学

微信：Rain-Splash 或 bubumelody 或 petrichoryin

QQ：1171374494

所有题目对应视频讲解需要付费，有需要的同学优先添加微信

顺序表默认结构体:

```
#define maxsize 50
typedef struct{
    int data[maxsize];
    int length;
```

```
} Sqlist;
```

1、顺序表递增有序，插入元素 x ，仍递增有序

```
int find_L(Sqlist L, int x){
    for (int i = 0; i < L.length; i++){
        if (x < L.data[i])
            return i;
    }
    return L.length;
}
```

```
void insert_L(Sqlist *L, int x){
    int pos = find_L(*L, x);
    for (int i = L->length - 1; i >= pos; i--)
        L->data[i + 1] = L->data[i];
    L->data[pos] = x;
    L->length++; //别遗漏
} //时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```

这是一个增和查的结合题目。因为是有序的，建议大家二轮复习的时候，也就是学完二分查找之后，再做一下这个题目。

2、将 n 个整数存放在数组 R 中，设计尽可能高效的算法，将 R 序列循环左移 p 个位置($0 < p < n$)，即将 R 中的数据由 $(X_0, X_1, \dots, X_{n-1})$ 变换为 $(X_p, X_{p+1}, \dots, X_{n-1}, X_0, X_1, \dots, X_{p-1})$ (2010 年统考题)

```
void Reverse(int R[], int start, int end){
    int i = start, j = end; //也可以写到 for 循环内
    for (; i < j; ++i, --j){
        int temp = R[i];
        R[i] = R[j];
        R[j] = temp;
    }
}
```

//要清楚函数每个参数的意思

```
void change(int R[], int n, int p){
    Reverse(R, 0, n - 1);
    Reverse(R, 0, n - p - 1);
    Reverse(R, n - p, n - 1);
} //时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```

6、尽可能高效地从顺序表中删除所有值重复的元素，仅保留第一个

```
void del(Sqlist *L){
    for (int i = 0; i < L->length; i++){
        int k = i + 1;
        for (int j = i + 1; j < L->length; j++)
            if (L->data[j] != L->data[i]){
                L->data[k++] = L->data[j];
            }
    }
}
```

//时空复杂度分别为 $O(N^2)$ 和 $O(1)$

7、尽可能高效地从有序表中删除所有值重复的元素，仅保留第一个

```
void del(Sqlist *L){
    int i = 0;
    int k = i + 1;
    for (int j = i + 1; j < L->length; j++)
        if (L->data[j] != L->data[i]){
            L->data[k++] = L->data[j];
            i++;
        }
    L->length = k;
```

} //时空复杂度分别为 $O(N)$ 和 $O(1)$

8、已知整数序列 $A = \{a_0, a_1, \dots, a_{n-1}\}$ 。其中 $0 \leq a_i < n$ ($0 \leq i < n$)。若一个整数序列中有过半相同元素，则称其为主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$ ，则 5 为主元素；又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$ ，则 A 没有主元素。设计尽可能高效的算法找出数组 A 中的主元素，若存在主元素则输出，否则输出 -1 (2013 年统考题)

```
int func(int A[], int n){
    int *B = (int *)malloc(sizeof(int) * n);
    for (int i = 0; i < n; i++)
        B[i] = 0;
    for (int i = 0; i < n; i++)
        B[A[i]]++;
    for (int i = 0; i < n; i++)
        if (B[i] > n/2)
            return i;
    return -1;
```

} //时空复杂度分别为 $O(N)$ 和 $O(N)$

9、一个含 n 个整数的数组，设计时间上尽可能高效的算法，找出数组中未出现的最小正整数。
例：{1,2,5}中未出现的最小正整数是 4。（18 年统考题）

```
int func(int A[], int n){
    int *B = (int *)malloc(sizeof(int)*(n+1));
    for (int i = 0; i < n+1; ++i)
        B[i] = 0;
    for (int i = 0; i < n; ++i)
        if (A[i] > 0 && A[i] <= n)
            B[A[i]]++;
    for (int i = 1; i < n+1; ++i)
        if (B[i] == 0)
            return i;
    return n+1;
}
```

//时空复杂度分别为 $O(N)$ 和 $O(N)$

10、两个递增有序表合并成一个递增有序表

```
void merge(Sqlist A, Sqlist B, Sqlist *C){
    int i = 0, j = 0, k = 0;
    while (i < A.length && j < B.length)
        if (A.data[i] < B.data[j])
            C->data[k++] = A.data[i++];
        else
            C->data[k++] = B.data[j++];
    while (i < A.length)
        C->data[k++] = A.data[i++];
    while (j < B.length)
        C->data[k++] = B.data[j++];
    C->length = A.length + B.length;
} //时空复杂度分别为  $O(M+N)$ 和  $O(1)$ 
```

11、两个递增有序表合并成一个递减有序表

```
void merge(Sqlist A, Sqlist B, Sqlist *C){
    int i = A.length-1, j = B.length-1, k = 0;
    while (i >= 0 && j >= 0)
        if (A.data[i] > B.data[j])
            C->data[k++] = A.data[i--];
        else
            C->data[k++] = B.data[j--];
    while (i >= 0)
        C->data[k++] = A.data[i--];
    while (j >= 0)
        C->data[k++] = B.data[j--];
    C->length = A.length + B.length;
} //时空复杂度分别为  $O(M+N)$ 和  $O(1)$ 
```

12、将数组中所有负数移动到数组最前端，所有非负数移动到数组的最后端

```
void func(int A[], int n){
    int left = 0, right = n-1;
    while (left < right){
        while (A[left] < 0 && left < right)
            left++;
        while (A[right] >= 0 && left < right)
            right--;
        if (left < right){
            int temp = A[left];
            A[left] = A[right];
            A[right] = temp;
        }
    }
}
```

如果奇数移动到最前端，偶数移动到最后端？

13、三个序列 **A, B, C** 长度均为 n ，且均为无重复元素的递增序列，设计一个尽可能高效的算法，输出三个序列中共同存在的所有元素（三指针）

```
int fmax(int a, int b, int c){
    if (a > b && a > c)
        return a;
    else if (b > a && b > c)
        return b;
    else
        return c;
}

void samekey(int A[], int B[], int C[], int n){
    int i = 0, j = 0, k = 0;
    while (i < n && j < n && k < n)
        if (A[i] == B[j] && A[i] == C[k]){
            printf("%d", A[i]);
            i++;
            j++;
            k++;
        }
        else{
            int max_num = fmax(A[i], B[j], C[k]);
            if (A[i] < max_num)
                i++;
            if (B[j] < max_num)
                j++;
            if (C[k] < max_num)
                k++;
        } //if else 是个整体，while 循环不用加括号
} //时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```

14、三元组(a,b,c)的距离 $D=|a-b|+|b-c|+|c-a|$ 。
给定 3 个非空整数集合 S1,S2 和 S3,按升序分别存储在 3 个数组中。设计一个尽可能高效的算法,计算所有可能的三元组(a,b,c)中的最小距离。如 S1={-1,0,9}, S2={-25,-10,10,11},S3={2,9,17,30,41},则最小距离为 2,对应的三元组为(9,10,9)。

(2020 年统考题)

```
int abs(int a){
    if (a < 0)
        return -a;
    else
        return a;
} //abs 函数, c 语言需要调用<stdlib.h>
int f1(int S1[], int m, int S2[], int n, int S3[], int p){
    int i = 0, j = 0, k = 0, min_dis = 1e5;
    while (i < m && j < n && k < p) {
        int dis = abs(S1[i] - S2[j]) + abs(S1[i] - S3[k])
            + abs(S2[j] - S3[k]);
        if (dis < min_dis)
            min_dis = dis;
        if (S1[i] <= S2[j] && S1[i] <= S3[k])
            i++;
        else if (S2[j] <= S1[i] && S2[j] <= S3[k])
            j++;
        else
            k++;
    }
    return min_dis;
} //时空复杂度分别为 O(N)和 O(1)
```

单链表默认结构体:

```
typedef struct LNode{
    int data;
    struct LNode *next;
} LNode, *LinkList;
```

1、试编写算法将带头结点的单链表就地逆置,即不需要借助辅助空间,保证空间复杂度为 O(1)

```
void reverse(LinkList L){
    LinkList p = L->next;
    L->next = NULL;
    LinkList r;
    while (p != NULL){
        r = p->next;
        p->next = L->next;
        L->next = p;
        p = r;
    }
} //时空复杂度分别为 O(N)和 O(1)
```

2、设有一个由正整数组成的无序单链表,编写程序之下以下功能:

- 1、找出最小值结点(非最后一个且唯一)
- 2、若该数值是奇数,将其于后继结点的结点交换(注意不是数值交换)
- 3、若该数值是偶数,则将其后继结点删除

```
bool func(LinkList L){
    LinkList p = L, pos=L;
    while (p->next != NULL){
        if (p->next->data < pos->next->data)
            pos = p;
        p = p->next;
    }
    if (pos->next->next == NULL)
        return false;
    LinkList u = pos->next;
    LinkList r = u->next;
    u->next = r->next;
    if (pos->next->data % 2 == 0)
        free(r);
    else{
        r->next = pos->next;
        pos->next = r;
    }
    return true;
} //时空复杂度分别为 O(N)和 O(1)
```

3、尽可能高效地删除递增链表中重复的结点，仅保留第一个。

```
void del(LinkList L){
    LinkList p = L->next;
    while (p->next != NULL){
        LinkList q = p->next;
        if (p->data == q->data){
            p->next = q->next;
            free(q);
        }
        else
            p = p->next;
    }
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(1)$

4、删除单链表中重复的结点，仅保留第一个

```
void del(LinkList L){
    LinkList p = L->next;
    while (p != NULL){
        LinkList r = p;
        while (r->next != NULL)
            if (r->next->data == p->data ){
                LinkList q = r->next;
                r->next = q->next;
                free(q);
            }
        else
            r = r->next;
        p = p->next;
    }
}
```

} //时空复杂度分别为 $O(N^2)$ 和 $O(1)$

5、用单链表保存 m 个整数，且 $|data| \leq n$ ，设计时间上尽可能高效的算法，对于 $data$ 绝对值相等的点，仅保留第一次出现的点（2015 统考题）

```
void del(LinkList L, int n){
    int *A = (int *)malloc( sizeof(int)* (n + 1) );
    for (int i = 0; i < n + 1; ++i)
        A[i] = 0;
    LinkList p = L;
    int m;
    while (p->next != NULL){
        if (p->next->data >= 0)
            m = p->next->data;
        else
            m = -p->next->data;
        if (A[m] == 0){
            A[m] = 1;
            p = p->next;
        }
        else{
            LinkList r = p->next;
            p->next = r->next;
            free(r);
        }
    }
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(N)$

6、将一个带头结点的单链表 A 分解成两个带头结点的单链表 A 和 B ，使 A 中含奇数位置元素， B 中含偶数位置元素，且相对位置不变

```
LinkList split(LinkList A){
    LinkList B = (LinkList)malloc(sizeof(LNode));
    LinkList rb = B, p = A;
    int pos = 1;
    while (p->next != NULL){
        if (pos % 2 == 1)
            p = p->next;
        else{
            LinkList r = p->next;
            p->next = r->next;
            rb->next = r;
            rb = r;
        }
        pos++;
    }
    rb->next = NULL;
    return B;
}
```

} //时空复杂度分别为 $O(N)$ 和 $O(1)$

7、将一个单链表{a1,b1,a2,b2.....an,bn}拆分成 { a1,a2.....an }和{ bn,bn-1,.....b1 }

```
LinkedList split(LinkedList A){
    LinkedList B = (LinkedList)malloc(sizeof(LNode));
    B->next = NULL;
    LinkedList p = A;
    int pos = 1;
    while (p->next != NULL){
        if (pos % 2 == 1)
            p = p->next;
        else{
            LinkedList r = p->next;
            p->next = r->next;
            r->next = B->next;
            B->next = r;
        }
        pos++;
    }
    return B;
}
```

8、两个递增有序的单链表，设计算法将 B 合并到 A 中，仍保证一个非递减有序的链表（双指针基础，很重要！）

```
void merge(LinkedList A, LinkedList B){
    LinkedList ra = A, p = A->next, q = B->next;
    while (p != NULL && q != NULL){
        if (p->data <= q->data){
            ra->next = p;
            ra = p;
            p = p->next;
        }
        else{
            ra->next = q;
            ra = q;
            q = q->next;
        }
    }
    if (p != NULL)
        ra->next = p;
    if (q != NULL)
        ra->next = q;
}
```

} //时空复杂度分别为 $O(M+N)$ 和 $O(1)$

其中 M 和 N 分别代表 A 和 B 链表的长度

9、两个递增有序的单链表，设计算法将 B 合并到 A 中，保证一个非递增有序的链表

```
void merge(LinkedList A, LinkedList B){
    LinkedList p = A->next, q = B->next, r;
    A->next = NULL;
    B->next = NULL;
    while (p != NULL && q != NULL){
        if (p->data <= q->data){
            r = p; //我的 r 在第一行定义了哦
            p = p->next;
        }
        else{
            r = q;
            q = q->next;
        }
        r->next = A->next;
        A->next = r;
    }
    while (p != NULL){
        r = p; p = p->next;
        r->next = A->next;
        A->next = r;
    }
    while (q != NULL){
        r = q; q = q->next;
        r->next = A->next;
        A->next = r;
    }
}
```

} //时空复杂度分别为 $O(M+N)$ 和 $O(1)$

10、A, B 两个单链表递增有序，从 A, B 中找出公共元素产生单链表 C，要求：不破坏 A 和 B 结点（交集）

```
LinkedList common(LinkedList A, LinkedList B){
    LinkedList C = (LinkedList)malloc(sizeof(LNode));
    LinkedList p = A->next, q = B->next, rc = C;
    while (p != NULL && q != NULL){
        if (p->data > q->data)
            q = q->next;
        else if (p->data < q->data)
            p = p->next;
        else{ //下面这行位置不够所以没对齐 hh
            LinkedList r = (LinkedList)malloc(sizeof(LNode));
            r->data = p->data;
            rc->next = r;
            rc = r;
            p = p->next;
            q = q->next;
        }
    }
    rc->next = NULL;
    return C;
}
```

} //时空复杂度分别为 $O(M+N)$ 和 $O(1)$

补充说明这里的空间复杂度为什么是 $O(1)$ 。虽然创建了很多链表结点，但是这是题目要求我们做的，不是我们为了达到目的自己创建的额外的空间。所以这道题空间复杂度就是 $O(1)$ 。不过考试一般不会出这种有歧义的题目，大家可以放心。

11、A, B 两个单链表递增有序，从 A, B 中找出公共元素存放到 A 链表中。要利用 A 链表的原有结点空间（交集）

```
LinkedList common(LinkedList A, LinkedList B){
    LinkedList p = A, q = B->next;
    while (p->next != NULL && q != NULL){
        if (p->next->data > q->data)
            q = q->next;
        else if (p->next->data < q->data){
            LinkedList r = p->next;
            p->next = r->next;
            free(r);
        }
        else{
            p = p->next;
            q = q->next;
        }
    }
    if (p->next != NULL)
        p->next = NULL;
} //时空复杂度分别为  $O(M+N)$  和  $O(1)$ 
```

12、假定采用带头结点的单链表保存单词，当两个单词有相同的后缀时，可共享相同的后缀存储空间。设 str1 和 str2 分别指向两个单词所在链表的头结点，请设计一个时间上尽可能高效的算法，找出这两个链表的共同后缀的起始位置。

(2012 年统考题)

```
LinkedList find(LinkedList str1, LinkedList str2){
    LinkedList p = str1->next, q = str2->next;
    int len1 = 0, len2 = 0;
    while (p != NULL){
        p = p->next;
        len1++;
    }
    while (q != NULL){
        q = q->next;
        len2++;
    }
    for (p = str1->next; len1 > len2; len1++)
        p = p->next;
    for (p = str2->next; len2 > len1; len1++)
        q = q->next;
    while(p != NULL && q != NULL){
        if (p == q)
            return p;
        p = p->next;
        q = q->next;
    }
    return NULL;
} //时空复杂度分别为  $O(\max(M,N))$  和  $O(1)$ 
```

13、查找单链表中倒数第 k 个结点, 若查找成功, 则输出该结点的 **data**, 并返回 1, 否则返回 0 (2009 年统考题)

//法一最优解 (双指针思想)

```
int find(LinkList L, int k){
```

```
    LinkList p = L, q = L;
```

```
    int i = 0;
```

```
    while (q != NULL){
```

```
        q = q->next;
```

```
        i++;
```

```
        if (i > k)
```

```
            p = p->next;
```

```
    }
```

```
    if (p == L || k <= 0)
```

```
        return 0;
```

```
    else{
```

```
        printf("%d", p->data);
```

```
        return 1;
```

```
    }
```

```
} //时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```

//法二暴力解

```
int len(LinkList L){
```

```
    LinkList p = L->next;
```

```
    int n = 0;
```

```
    while (p != NULL){
```

```
        p = p->next;
```

```
        n++;
```

```
    }
```

```
    return n;
```

```
} //先统计单链表长度
```

```
int find(LinkList L, int k){
```

```
    if (k > len(L) || k <= 0)
```

```
        return 0;
```

```
    int m = len(L) - k + 1;
```

```
    int i = 1;
```

```
    LinkList p = L->next;
```

```
    while (i < m) {
```

```
        p = p->next;
```

```
        i++;
```

```
    }
```

```
    printf("%d", p->data);
```

```
    return 1;
```

```
} //时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```

14、给定一个单链表 $L(a_1, a_2, a_3, \dots, a_n)$, 将其重新排列为 $(a_1, a_n, a_2, a_{n-1}, \dots)$ (2019 统考题)

```
void func(LinkList L){
```

```
    LinkList slow = L, fast = L;
```

```
    while (fast != NULL && fast->next != NULL){
```

```
        slow = slow->next;
```

```
        fast = fast->next->next;
```

```
    }
```

```
    LinkList p = slow->next;
```

```
    slow->next = NULL;
```

```
    while (p != NULL){
```

```
        LinkList r = p->next;
```

```
        p->next = slow->next;
```

```
        slow->next = p;
```

```
        p = r;
```

```
    } //逆置
```

```
    LinkList mid = slow->next;
```

```
    slow->next = NULL;
```

```
    LinkList beg = L->next;
```

```
    while (mid != NULL){
```

```
        LinkList r = mid->next;
```

```
        mid->next = beg->next;
```

```
        beg->next = mid;
```

```
        beg = mid->next;
```

```
        mid = r;
```

```
    }
```

```
} //时空复杂度分别为  $O(N)$ 和  $O(1)$ 
```


二叉树的结构体 (链式存储) (二叉链表存储)

```
typedef struct BTNode{
    char data;
    struct BTNode *lchild, *rchild;
} BTNode, *BiTree;
```

1、找出二叉树中值最大的结点

```
void func(BiTree p, BiTree *m){
    if (p != NULL){
        if ((*m) == NULL || p->data > (*m)->data)
            *m = p;
        func(p->lchild, m);
        func(p->rchild, m);
    }
} //时空复杂度分别为 O(N) 和 O(N)
```

2、把二叉树所有结点左右子树交换

```
void swap(BiTree p){
    if (p != NULL){
        BiTree temp = p->lchild;
        p->lchild = p->rchild;
        p->rchild = temp;
        swap(p->lchild);
        swap(p->rchild);
    }
} //时空复杂度分别为 O(N) 和 O(N)
```

3、判断二叉树是不是正则二叉树 (即每个结点的度均为 0 或 2)

计算型:

```
int func(BiTree p){
    if (p == NULL)
        return 1;
    if ((p->lchild == NULL && p->rchild != NULL) ||
        (p->lchild != NULL && p->rchild == NULL))
        return 0;
    int L1 = func(p->lchild);
    int L2 = func(p->rchild);
    return L1 && L2;
}
```

操作型:

```
void func(BiTree p, int *flag){
    if (p != NULL){
        if ((p->lchild == NULL && p->rchild != NULL) ||
            (p->lchild != NULL && p->rchild == NULL))
            *flag = 0;
        func(p->lchild, flag);
        func(p->rchild, flag);
    }
} //时空复杂度分别为 O(N) 和 O(N)
```

4、层次遍历二叉树

```
void level(BiTree bt){
    BiTree que[maxsize];
    int front = 0, rear = 0;
    if (bt != NULL){
        que[++rear] = bt;
        while (front != rear){
            bt = que[++front];
            printf("%d", bt->data);
            if (bt->lchild != NULL)
                que[++rear] = bt->lchild;
            if (bt->rchild != NULL)
                que[++rear] = bt->rchild;
        }
    }
}
```

} //时空复杂度分别为 O(N) 和 O(N)

//这里入队和出队的时候, 先++再赋值或者先赋值再++都是可以的, rear 和 front 保持一致即可

需要讲义对应讲解视频, 可加下方微信获取



5、求树的度（树中结点度取最大值为树的度）

```
void func(BiTree p, int *max_degree){
```

```
    if (p != NULL){
        int degree;
        if (p->lchild && p->rchild)
            degree = 2;
        else if (p->lchild || p->rchild)
            degree = 1;
        else
            degree = 0;
        if (degree > *max_degree)
            *max_degree = degree;
        func(p->lchild, max_degree);
        func(p->rchild, max_degree);
    }
```

```
} //操作型
```

```
//时空复杂度分别为 O(N)和 O(N)
```

```
int func(BiTree p){
```

```
    if (p == NULL || (!p->lchild && !p->rchild))
        return 0;
    if (p->lchild && p->rchild)
        return 2;
    if (p->lchild || p->rchild){
        int A = func(p->lchild);
        int B = func(p->rchild);
        int max_degree = A > B ? A : B;
        return max_degree > 1 ? max_degree : 1;
    }
```

```
} //计算型
```

6、判断两个二叉树是否相似（相等）

```
int func(BiTree T1, BiTree T2){
```

```
    if (T1 == NULL && T2 == NULL)
        return 1;
    if (T1 == NULL || T2 == NULL)
        return 0;
    if (T1->data != T2->data)
        return 0; //本语句在判断两树相等时再加
    int left = func(T1->lchild, T2->lchild);
    int right = func(T1->rchild, T2->rchild);
    return left && right;
} //时空复杂度分别为 O(N)和 O(N)
```

7、将给定的二叉树转化为等价的中缀表达式(具体细节图在视频中会提到) (2017 年统考题)

```
void func(BiTree p, int *deep){
```

```
    if (p != NULL){
        ++(*deep);
        if ((p->lchild && p->rchild) && *deep > 1)
            printf("(");
        func(p->lchild, deep);
        printf("%c", p->data);
        func(p->rchild, deep);
        if ((p->lchild && p->rchild) && *deep > 1)
            printf(")");
        --(*deep);
    }
```

```
} //时空复杂度分别为 O(N)和 O(N)
```

8、 $(a-(b+c))*(d/e)$ 存储在二叉树(图在视频里出现)，求四则运算后的结果

```
int func(BiTree p){
```

```
    if (p == NULL)
        return 0;
    if (!p->lchild && !p->rchild)
        return p->data - '0';
    if (p->lchild && p->rchild){
        int A = func(p->lchild);
        int B = func(p->rchild);
        if (p->data == '+')
            return A + B;
        else if (p->data == '-')
            return A - B;
        else if (p->data == '*')
            return A * B;
        else if (p->data == '/')
            return A / B;
    }
```

```
} //时空复杂度分别为 O(N)和 O(N)
```

9、输出根结点到每个叶子结点的路径

```
void allpath(BiTree p, char st[], int *top){
    if (p != NULL){
        st[++(*top)] = p->data;
        if (!p->lchild && !p->rchild){
            for (int i = 0; i <= *top; i++)
                printf("%c", pathstack[i]);
        }
        allpath(p->lchild, st, top);
        allpath(p->rchild, st, top);
        --(*top);
    }
}

//根结点到每个叶子点的顺序
//时空复杂度分别为 O(N)和 O(N)

void allpath(BiTree p, char st[], int* top){
    if (p != NULL){
        st[++(*top)] = p->data;
        if (!p->lchild && !p->rchild){
            for (int i = *top; i >= 0 ; i--)
                printf("%c", st[i]);
        }
        allpath(p->lchild, st, top);
        allpath(p->rchild, st, top);
        (*top)--;
    }
}

//每个叶子到根结点的顺序
如果是根结点到值为 x 结点的路径呢？根结点到所有结点的路径呢？均为祖先问题，直接秒杀！
```

10、使用递归方法输出根结点到叶子结点最长的一条路径。

```
void allpath(BiTree p, char st[], int *top, int *max,
char res[]) {
    if (p != NULL){
        st[++(*top)] = p->data;
        if (!p->lchild && !p->rchild) {
            if (*top > *max) {
                *max = *top;
                for (int i = 0; i <= *top; i++)
                    res[i] = st[i];
            }
        }
        allpath(p->lchild, st, top, max, res);
        allpath(p->rchild, st, top, max, res);
        (*top)--;
    }
}

//时空复杂度分别为 O(N)和 O(N)
```

11、找到 p 和 q 最近公共祖先结点 r

```
BiTree find_LCA(BiTree root, BiTree p, BiTree q) {
    if (root==NULL || root == p || root == q)
        return root;
    BiTree left = find_LCA(root->lchild, p, q);
    BiTree right = find_LCA(root->rchild, p, q);
    if (left == NULL && right == NULL)
        return NULL;
    if (left != NULL && right != NULL)
        return root;
    return left != NULL ? left : right;
}

//递归思想
//时空复杂度分别为 O(N)和 O(N)

如果二叉树顺序存储呢？（408 必会）

BiTree find_LCA(BiTree aa[], int i, int j){
    while (i != j){
        if (j > i)
            j = (j - 1) / 2;
        else
            i = (i - 1) / 2;
    }
    return aa[i];
}

//时空复杂度分别为 O(logN)和 O(1)
```

12、层次遍历求解二叉树的高度

```
void level(BiTree bt){
    BiTree que[maxsize];
    int front = 0, rear = 0, last = 1, level = 0;
    if (bt != NULL){
        que[++rear] = bt;
        while (front != rear){
            bt = que[++front];
            if (bt->lchild != NULL)
                que[++rear] = bt->lchild;
            if (bt->rchild != NULL)
                que[++rear] = bt->rchild;
            if (front == last){
                level++;
                last = rear;
            }
        }
    }
    return level;
}
//时空复杂度分别为 O(N)和 O(N)
```

拓展：层次遍历求解二叉树中值为 x 的高度？

13、计算二叉树的带权路径长度（叶子结点）

```
int func(BiTree bt){
    BiTree que[maxsize];
    int front=0, rear=0, wpl=0, last = 1, level = 0;
    if (bt != NULL){
        que[++rear] = bt;
        while (front != rear){
            bt = que[++front];
            if (!bt->lchild && !bt->rchild)
                wpl += level * (bt->data);
            if (bt->lchild != NULL)
                que[++rear] = bt->lchild;
            if (bt->rchild != NULL)
                que[++rear] = bt->rchild;
            if (front == last){
                level++;
                last = rear;
            }
        }
    }
    return wpl;
}
//法一，层次遍历思想
//时空复杂度分别为 O(N)和 O(N)
```

递归遍历也要会写！

14、求解二叉树的宽度

```
int level(BiTree bt){
    BiTree que[maxsize];
    int front=0, rear=0, res=0, last=1, width=0;
    if (bt != NULL){
        que[++rear] = bt;
        while (front != rear){
            bt = que[++front];
            res++;
            if (bt->lchild != NULL)
                que[++rear] = bt->lchild;
            if (bt->rchild != NULL)
                que[++rear] = bt->rchild;
            if (front == last){
                if (res > width)
                    width = res;
                res=0;
                last = rear;
            }
        }
    }
    return width;
}
//法一，层次遍历思想
//时空复杂度分别为 O(N)和 O(N)
```

```

void func(BiTree bt, int A[], int *L){
    if (bt != NULL){
        ++(*L)
        A[*L]++;
        func(bt->lchild, A, L);
        func(bt->rchild, A, L);
        --(*L)
    }
}

int max_width(BiTree bt){
    int A[maxsize] = {0},
    int L = 0;
    func(T, A, &L);
    int width = 0;
    for (int i = 1; i < maxsize; i++)
        if (A[i] > width)
            width = A[i];
    return width;
} //法二，递归遍历思想

```

15、判断二叉树是否为完全二叉树

```

bool fun(BiTree bt){
    BiTree que[maxsize];
    int front = 0, rear = 0;
    if (bt != NULL){
        que[++rear] = bt;
        while (front != rear){
            bt = que[++front];
            if (bt != NULL){
                que[++rear] = bt->lchild;
                que[++rear] = bt->rchild;
            }
        }
        else{
            while (front != rear)
                if (que[++front] != NULL)
                    return false;
        }
    }
    return true;
} //时空复杂度分别为 O(N)和 O(N)

```

拓展：判断二叉树是否为满二叉树

```

void func(BiTree p, int *n, int *L, int *max_L){
    if (p != NULL){
        ++(*L);
        if ((*L) >= (*max_L))
            *max_L = *L;
        ++(*n);
        func(p->lchild, n, L, max_L);
        func(p->rchild, n, L, max_L);
        --(*L);
    }
}

bool Is_True(BiTree p){
    int n=0, L=0, max_L = 0;
    func(p, &n, &L, &max_L);
    if (pow(2, max_L) - 1 == n)
        return true;
    else
        return false;
}

```

18、 二叉树以二叉链表表示，设计算法存储到一维数组中

```
void create(BiTree p, char A[], int i){
    if (p != NULL){
        A[i] = p->data;
        create(p->lchild, A, 2 * i + 1);
        create(p->rchild, A, 2 * i + 2);
    }
}
```

}注意这是第 19 题，先看完 17 和 18 再看这个
//时空复杂度分别为 $O(N)$ 和 $O(N)$

16、 满二叉树先序序列存在于数组中，设计算法将其变成后序序列

```
void preTopost(char pre[], int L1, int R1, char post[], int L2, int R2){
    if (L1 <= R1){
        post[R2] = pre[L1];
        preTopost(pre, L1 + 1, (L1 + R1 + 1)/2, post, L2, (L2 + R2 - 1)/2);
        preTopost(pre, (L1 + R1 + 1)/2 + 1, R1, post, (L2 + R2 - 1)/2 + 1, R2 - 1);
    }
}
```

}//先序转后序

```
void postTopre(char post[], int L1, int R1, char pre[], int L2, int R2){
    if (L1 <= R1){
        pre[L2] = post[R1];
        postTopre(post, L1, (L1 + R1 - 1), pre, L2 + 1, (L2 + R2 + 1)/2);
        postTopre(post, (L1 + R1 - 1)/2 + 1, R1 - 1, pre, (L2 + R2 + 1)/2 + 1, R2);
    }
}
```

}//后序转先序!

//时空复杂度分别为 $O(N)$ 和 $O(N)$

17、 先序与中序遍历分别存在两个一维数组 **A, B** 中，试建立二叉链表

```
BiTree func(char pre[], char mid[], int L1, int R1, int L2, int R2){
    BiTree root = (BiTree)malloc(sizeof(BTNode));
    root->data = pre[L1];
    int i = L2;
    for (; mid[i] != pre[L1]; i++); //注意 for 循环内无内容
    if (i > L2)
        root->lchild = func(pre, mid, L1 + 1, L1 + i - L2, L2, i - 1);
    else
        root->lchild = NULL;
    if (i < R2)
        root->rchild = func(pre, mid, L1 + i - L2 + 1, R1, i + 1, R2);
    else
        root->rchild = NULL;
    return root;
}
```

}//先序和中序构建二叉链表

```
BiTree func(char post[], char mid[], int L1, int R1, int L2, int R2){
    BiTree root = (BiTree)malloc(sizeof(BTNode));
    root->data = post[R1];
    int i = L2;
    for (; mid[i] != post[R1]; i++); //注意 for 循环内无内容
    if (i > L2)
        root->lchild = func(post, mid, L1, i - L2 + L1 - 1, L2, i - 1);
    else
        root->lchild = NULL;
    if (i < R2)
        root->rchild = func(post, mid, i - L2 + L1, R1 - 1, i + 1, R2);
    else
        root->rchild = NULL;
    return root;
}
```

}//后序和中序构建二叉链表 //时空复杂度分别为 $O(N)$ 和 $O(N)$

18 题拓展：二叉树以顺序方式存在于数组 **A** 的中，设计算法以二叉链表表示

```
BiTree create(char A[], int i, int n){
```

```
    if (i < n && A[i] != '#'){
```

```
        BiTree t = (BiTree)malloc(sizeof(BTNode));
```

```
        t->data = A[i];
```

```
        t->lchild = create(A, 2 * i + 1, n);
```

```
        t->rchild = create(A, 2 * i + 2, n);
```

```
        return t;
```

```
    }
```

```
    return NULL;
```

```
} //时空复杂度分别为 O(N)和 O(N)
```

19、已知二叉树采用顺序存储结构，编写算法统计该树中所有叶子结点个数。

```
int count(char A[], int i, int N) {
```

```
    if (i >= N || A[i] == '#')
```

```
        return 0;
```

```
    if ((2 * i + 1 >= N || A[2 * i + 1] == '#') &&
```

```
        (2 * i + 2 >= N || A[2 * i + 2] == '#'))
```

```
        return 1;
```

```
    int n1 = count(A, 2 * i + 1, N);
```

```
    int n2 = count(A, 2 * i + 2, N);
```

```
    return n1 + n2;
```

```
} //时空复杂度分别为 O(N)和 O(N)
```

//408 近几年爱考顺序存储的树和图,务必掌握!

图的结构体

邻接表存储:

```
typedef struct ANode{
```

```
    int adjvex; //边所指向结点的位置
```

```
    struct ANode *nextarc;
```

```
} ANode, *Node; //边结点结构体
```

```
typedef struct{
```

```
    int data;
```

```
    ANode *firstarc;
```

```
} Vnode; //顶点结构体
```

```
typedef struct{
```

```
    int numver, numedg;
```

```
    Vnode adjlist[maxsize];
```

```
} Graph;
```

邻接矩阵存储:

```
typedef struct{
```

```
    int numver, numedg;
```

```
    int vertice[maxsize];
```

```
    int Edge[maxsize][maxsize];
```

```
} mGraph;
```

1、邻接表实现图的广度优先遍历 (BFS)

```
void BFS(Graph G, int v){
```

```
    int visit[maxsize] = {0};
```

```
    int que[maxsize];
```

```
    int front = 0, rear = 0;
```

```
    visit[v] = 1;
```

```
    que[++rear] = v;
```

```
    while (front != rear){
```

```
        v = que[++front];
```

```
        printf("%d", v);
```

```
        Node p = G.adjlist[v].firstarc;
```

```
        for( ; p != NULL; p = p->nextarc)
```

```
            if (visit[p->adjvex] == 0){
```

```
                visit[p->adjvex] = 1;
```

```
                que[++rear] = p->adjvex;
```

```
            }
```

```
        }
```

} //p 的初始化最好写到 for 循环里面, 这里主要是一行盛不下了哈哈~

2、邻接矩阵实现图的广度优先遍历 (BFS)

```
void BFS(MGraph G, int v){
    int visit[maxsize] = {0};
    int que[maxsize];
    int front = 0, rear = 0;
    que[++rear] = v;
    visit[v] = 1; //本句写在入队前或入队后都可
    while (front != rear){
        v = que[++front];
        printf("%d", v);
        for (int i=0; i<G.numver; i++){
            if (G.Edge[v][i] == 1 && visit[i] == 0){
                que[++rear] = i;
                visit[i] = 1;
            }
        }
    }
} //时空复杂度分别为  $O(V^2)$ 和  $O(V)$ 
```

3、设计算法，求无向连通图距顶点 v 最远的一个结点（即路径长度最大）

```
int BFS(Graph G, int v){
    int visit[maxsize] = {0};
    int que[maxsize];
    int front = 0, rear = 0;
    visit[v] = 1;
    que[++rear] = v;
    while (front != rear){
        v = que[++front];
        Node p = G.adjlist[v].firstarc;
        for( ; p != NULL; p = p->nextarc)
            if (visit[p->adjvex] == 0){
                visit[p->adjvex] = 1;
                que[++rear] = p->adjvex;
            }
    }
    return v;
} //时空复杂度分别为  $O(V+E)$ 和  $O(V)$ 
```

4、邻接表实现图的深度优先遍历 (DFS)

```
void DFS(Graph G, int v, int visit[]){
    printf("%d", v);
    visit[v] = 1;
    Node p = G.adjlist[v].firstarc;
    for( ; p != NULL; p = p->nextarc)
        if (visit[p->adjvex] == 0)
            DFS(G, p->adjvex, visit);
} //时空复杂度分别为  $O(V+E)$ 和  $O(V)$ 
```

5、邻接矩阵实现图的深度优先遍历 (DFS)

```
void DFS(MGraph G, int v, int visit[]){
    printf("%d", v);
    visit[v] = 1;
    for (int i = 0; i < G.numver; i++)
        if (G.Edge[v][i] == 1 && visit[i] == 0)
            DFS(G, i, visit);
} //时空复杂度分别为  $O(V^2)$ 和  $O(V)$ 
```

6、有向图采用邻接表存储，设计算法判断顶点 Vi 和顶点 Vj 之间是否存在路径

```
void f1(Graph G, int i, int j, int visit[], bool* res){
    if (i == j)
        *res = true;
    visit[i] = 1;
    Node p = G.adjlist[i].firstarc;
    for( ; p != NULL; p = p->nextarc)
        if (visit[p->adjvex] == 0)
            f1(G, p->adjvex, j, visit, res);
} //法一：DFS
```



```

bool f2(Graph G, int i, int j){
    int visit[maxsize] = {0};
    int que[maxsize];
    int front = 0, rear = 0;
    que[rear++] = i;
    visit[i] = 1;
    while (front != rear){
        i = que[front++];
        if (i == j)
            return true;
        Node p = G.adjlist[i].firstarc;
        for (; p != NULL; p = p->nextarc){
            if(visit[p->adjvex] == 0){
                que[rear++] = p->adjvex;
                visit[p->adjvex] = 1;
            }
        }
    }
    return false;
}

```

法二：BFS

//拓展：有向图采用邻接矩阵存储呢？

//这种类型的题目也可以直接从 i 结点进行 dfs 或者 bfs 后，观察 visit[j]是否为 1。本质上一样。

7、在有向图中，如果顶点 r 到图中所有顶点都存在路径，则称 r 为图的根结点。编写代码输出有向图中所有根结点。

```

void func(Graph G){
    for(int i = 0; i < G.numver; i++){
        int visit[maxsize] = {0};
        int flag = 0;
        DFS(G, i, visit);
        for (int j = 0; j < G.numver; j++){
            if (visit[j] == 0)
                flag = 1;
        }
        if (flag == 0)
            printf("%d", i);
    }
}
//时空复杂度分别为  $O(V*(V+E))$ 和  $O(V)$ 

```

8、求无向图的连通分量个数

```

int func(Graph G){
    int visit[maxsize] = {0};
    int count = 0;
    for (int i = 0; i < G.numver; ++i)
        if (visit[i] == 0){
            DFS(G, i, visit);
            count++;
        }
    return count;
}
//时空复杂度分别为  $O(V*(V+E))$ 和  $O(V)$ 

```

9、已知无向连通图 G 由顶点集 V 和边集 E 组成， $|E|>0$ ，当 G 中度为奇数的顶点个数为不大于 2 的偶数时， G 存在包含所有边且长度为 $|E|$ 的路径（称为 EL 路径）。设图 G 采用邻接矩阵存储，设计算法判断图中是否存在 EL 路径，若存在返回 1，否则返回 0。（2021 年统考题）

```

int IsExistEL(MGraph G){
    int count = 0;
    for (int i = 0; i < G.numver; i++){
        int degree = 0;
        for (int j = 0; j < G.numver; j++){
            degree += G.Edge[i][j];
            if (degree % 2 != 0)
                count++;
        }
        if (count == 0 || count == 2)
            return 1;
        else
            return 0;
    }
}

```

//时空复杂度分别为 $O(V^2)$ 和 $O(1)$

//408 注意，结构体要和题目给出的保持一致

10、有向图 **G** 采用邻接矩阵存储，将图中出度大于入度的顶点称为 **K** 顶点。要求输出 **G** 中所有的 **K** 顶点，并返回 **K** 顶点的个数。（2023 年统考题）

```
int func(MGraph G){
    int count = 0;
    for(int i = 0; i < G.numver; i++){
        int indegree = 0, outdegree = 0;
        for (int j = 0; j < G.numver; j++){
            outdegree += G.Edge[i][j];
            indegree += G.Edge[j][i];
        }
        if (outdegree > indegree){
            printf("%c", G.verticle[i]);
            count++;
        }
    }
    return count;
} //时空复杂度分别为  $O(V^2)$  和  $O(1)$ 
```

11、有向图采用邻接表存储，计算每个顶点的入度和出度，将其存储到两个数组中

```
void func(Graph G, int inres[], int outres[]){
    for(int i = 0; i < G.numver; i++){
        inres[i] = 0;
        outres[i] = 0;
    }
    for (int i = 0; i < G.numver; i++){
        Node p = G.adjlist[i].firstarc;
        for(; p != NULL; p = p->nextarc){
            outres[i]++;
            inres[p->adjvex]++;
        }
    }
} //时空复杂度分别为  $O(V+E)$  和  $O(V)$ 
```

12、设计算法判断无向图是否是一棵树

```
void DFS(Graph G, int v, int visit[], int *vn){
    visit[v] = 1;
    ++(*vn);
    Node p = G.adjlist[v].firstarc;
    for(; p != NULL; p = p->nextarc){
        if (visit[p->adjvex] == 0)
            func(G, p->adjvex, visit, vn);
    }
} //时空复杂度分别为  $O(V+E)$  和  $O(V)$ 

bool IsTree(Graph G){
    int v = 0, vn = 0;
    int visit[maxsize] = {0};
    DFS(G, v, visit, &vn);
    if (vn == G.numver && (G.numver-G.numedg) == 1)
        return true;
    else
        return false;
}
```

13、输出有向图 V_i 顶点到 V_j 顶点所有简单路径。

```
void DFS(Graph G, int i, int j, int visit[], int path[],
        int *top){
    visit[i] = 1;
    path[++(*top)] = i;
    if (i == j){
        for(int k = 0; k < *top; k++){
            printf("%d ", path[k]);
        }
        Node p = G.adjlist[i].firstarc;
        for( ; p != NULL; p = p->nextarc)
            if (visit[p->adjvex] == 0)
                DFS(G, p->adjvex, j, visit, path, top);
        --(*top);
        visit[i] = 0;
    }
} //时空复杂度分别为  $O(V+E)$ 和  $O(V)$ 
```

拓展：输出 V_i 到 V_j 之间长度为 k 的简单路径？

14、拓扑排序判断有/无向图是否有环（邻接表）

```
bool tpsort (Graph G){
    int inres[G.numver]={0},
    int st[G.numver], top = -1, n = 0;
    for (int i = 0; i < G.numver; i++){
        Node p = G.adjlist[i].firstarc;
        for ( ; p != NULL; p = p->nextarc)
            inres[p->adjvex]++;
    }
    for (int i = 0; i < G.numver; i++)
        if (inres[i] == 0)//无向图为 1
            st[++top] = i;
    while (top != -1){
        int v = st[top--];
        n++;
        Node p = G.adjlist[v].firstarc;
        for ( ; p != NULL; p = p->nextarc)
            if (--inres[p->adjvex] == 0) //无向图为 1
                st[++top] = p->adjvex;
    }
    if (n == G.numver)
        return false; //无环
    else
        return true; //有环
} //时空复杂度分别为  $O(V+E)$ 和  $O(V)$ 
```

15、判断邻接矩阵存储的有向图的拓扑排序是否唯一（2024 年统考题）

```
bool tpsort(MGraph G) {
    int inres[G.numver] = {0};
    int st[G.numver], top = -1, n = 0;
    for (int i = 0; i < G.numver; i++)
        for (int j = 0; j < G.numver; j++)
            if (G.Edge[j][i] != 0)
                inres[i]++;
    for (int i = 0; i < G.numver; ++i)
        if (inres[i] == 0)
            st[++top] = i;
    if (top > 0)
        return false;
    while (top != -1) {
        int v = st[top--];
        n++;
        for (int i = 0; i < G.numver; i++)
            if (G.Edge[v][i] != 0 && (--inres[i] == 0))
                st[++top] = i;
        if (top > 0)
            return false;
    }
    if (n == G.numver)
        return true; //无环且唯一
    else
        return false; //有环
} //时空复杂度分别为  $O(V^2)$ 和  $O(V)$ 
```

1、在有序表中二分查找值为 **key** 的元素

```
int binsearch(SqList L, int key){
    int low = 0, high = L.length - 1, mid;
    while (low <= high){
        mid = (low + high) / 2;
        if (L.data[mid] == key)
            return mid + 1;
        else if (L.data[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
} //非递归，时空复杂度分别为  $O(\log N)$  和  $O(1)$ 
```

```
int func(SqList L, int key, int low, int high){
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
    if (L.data[mid] == key)
        return mid + 1;
    else if (L.data[mid] < key)
        return func(L, key, mid + 1, high);
    else
        return func(L, key, low, mid - 1);
}
```

//递归，时空复杂度分别为 $O(\log N)$ 和 $O(\log N)$
由于每次递归调用消耗的栈空间与递归深度成正比，因此空间复杂度也是 $O(\log N)$

2、判断给定二叉树是否是二叉(搜索)排序树

```
void inprint(BiTree T, int res[], int *index) {
    if (T != NULL) {
        inprint(T->lchild, res, index);
        res[(*index)++] = T->data;
        inprint(T->rchild, res, index);
    }
}
```

```
bool isture(int res[], int index){
    for(int i = 0; i < index-1; i++)
        if(res[i] >= res[i+1])
            return false;
    return true;
}
```

//法一，时空复杂度分别为 $O(N)$ 和 $O(N)$

```
bool func(BiTree T, int low, int high){
    if(T == NULL)
        return true;
    if(T->data <= low || T->data >= high)
        return false;
    bool left = func(T->lchild, low, T->data);
    bool right = func(T->rchild, T->data, high);
    return left && right;
}
```

//法二，时空复杂度分别为 $O(N)$ 和 $O(N)$

3、判断顺序存储的二叉树是否为二叉搜索树

```
bool func(SqBiTree T, int i, Elemtype low, Elemtype high){
    if(i >= T.ElemNum || T.Node[i] == -1)
        return true;
    if(T.Node[i] <= low || T.Node[i] >= high)
        return false;
    bool left = func(T, 2 * i + 1, low, T.Node[i]);
    bool right = func(T, 2 * i + 2, T.Node[i], high);
    return left && right;
}
```

//时空复杂度分别为 $O(N)$ 和 $O(N)$

4、判断一个二叉排序树是否为平衡二叉树

```
int height(BiTree T) {
    if (T == NULL)
        return 0;
    int A = height(T->lchild);
    int B = height(T->rchild);
    return (A > B ? A : B) + 1;
} //计算高度
bool istrue(BiTree T) {
    if (T == NULL)
        return true;
    int left = height(T->lchild);
    int right = height(T->rchild);
    if (abs(A - B) > 1)
        return false;
    return istrue(T->lchild) && istrue(T->rchild);
}
```

//时空复杂度分别为 $O(N)$ 和 $O(N)$

拓展：如果是顺序存储？不会写重新听第 3 题

1、直接插入排序

```
void InsertSort(int A[], int n){
    int j;
    for (int i = 2; i <= n; i++){
        A[0] = A[i];
        for (j = i - 1; A[0] < A[j]; j--)
            A[j+1] = A[j];
        A[j+1] = A[0];
    }
} //时空复杂度分别为  $O(N^2)$ 和  $O(1)$ 
```

2、折半插入排序

```
void BinInsert(int A[], int n){
    int j, low, high, mid;
    for (int i = 2; i <= n; i++){
        A[0] = A[i];
        low = 1;
        high = i-1;
        while (low <= high){
            mid = (low + high) / 2;
            if (A[mid] > A[0])
                high = mid - 1;
            else
                low = mid + 1;
        }
        for (j = i-1; j >= low; j--)
            A[j+1] = A[j];
        A[low] = A[0];
    }
} //时空复杂度分别为  $O(N^2)$ 和  $O(1)$ 
```

3、快速排序

```
int Partition(int A[], int low, int high){
    int pivot = A[low];
    while (low < high){
        while (low < high && A[high] >= pivot)
            high--;
        A[low] = A[high];
        while (low < high && A[low] < pivot)
            low++;
        A[high] = A[low];
    }
    A[low] = pivot;
    return low;
}

void QuickSort(int A[], int low, int high){
    if (low < high){
        int pos = Partition(A, low, high);
        QuickSort(A, low, pos - 1);
        QuickSort(A, pos + 1, high);
    }
} //时空复杂度分别为  $O(N\log N)$ 和  $O(\log N)$ 
```

4、二路归并排序

```
void merge(int A[], int low, int mid, int high){
    int *B = (int*)malloc((high + 1)*sizeof(int));
    for (int i = low; i <= high; i++)
        B[i] = A[i];
    int i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high)
        if (B[i] <= B[j])
            A[k++] = B[i++];
        else
            A[k++] = B[j++];
    while (i <= mid)
        A[k++] = B[i++];
    while (j <= high)
        A[k++] = B[j++];
}

void mergeSort(int A[], int low, int high){
    if (low < high){
        int mid = (low + high) / 2;
        mergeSort(A, low, mid);
        mergeSort(A, mid + 1, high);
        merge(A, low, mid, high);
    }
} //时空复杂度分别为  $O(N\log N)$ 和  $O(N)$ 
```

6、堆排序

```
void sift(int A[], int low, int high){
    int i = low, j = 2 * i + 1;
    int temp = arr[i];
    while (j <= high){
        if (j < high && A[j] < A[j + 1])
            ++j;
        if (temp < A[j]){
            A[i] = A[j];
            i = j;
            j = 2 * i + 1;
        }
        else
            break;
    }
    A[i] = temp;
} //堆调整的时空复杂度分别为  $O(\log N)$  和  $O(1)$ 

void heapSort(int A[], int n){
    for (int i = n / 2 - 1; i >= 0; --i)
        sift(A, i, n - 1);
    for (int i = n - 1; i > 0; --i){
        int temp = A[0];
        A[0] = A[i];
        A[i] = temp;
        sift(A, 0, i - 1);
    }
} //堆排序时空复杂度分别为  $O(N \log N)$  和  $O(1)$ 
```

5、判断数组是否是小根堆

```
bool isMinHeap(int A[], int n) {
    for (int i = 0; i <= (n - 2) / 2; i++) {
        if (A[i] > A[2 * i + 1])
            return false;
        if (2 * i + 2 < n && A[i] > A[2 * i + 2])
            return false;
    }
    return true;
} //时空复杂度分别为  $O(N)$  和  $O(1)$ 
```

7、已知由 $n(n \geq 2)$ 个正整数构成的集合 $A = \{a_k | 0 \leq k < n\}$ ，将其划分为两个不相交的子集 A_1 和 A_2 ，元素个数分别是 n_1 和 n_2 。 A_1 和 A_2 中元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法，满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。
(2016 年统考题)

```
void func(int A[], int A1[], int A2[], int n){
    Quicksort(A, 0, n - 1); //考试时快排代码要写
    for (int i = 0; i < n / 2; ++i)
        A1[i] = A[i];
    for (int i = n / 2; i < n; ++i)
        A2[i - n / 2] = A[i];
} //时空复杂度分别为  $O(N \log N)$  和  $O(\log N)$ 

最优解有一定难度，听不懂就暴力吧
```